arXiv:2308.16824v2 [cs.CL] 3 Sep 2023

CAN PROGRAMMING LANGUAGES BOOST EACH OTHER VIA INSTRUCTION TUNING?

TECHNICAL REPORT

Daoguang Zan[†*] **Ailun Yu**[§*] **Bo Shen**[‡] **Jiaxin Zhang**[‡] **Taihong Chen**[‡] **Bing Geng**[‡] **Bei Chen**[¶]
Jichuan Ji[‡] **Yafen Yao**[‡] **Yongji Wang**[†] **Qianxiang Wang**[‡]
[†]Institute of Software, Chinese Academy of Science
[§]Peking University
[‡]Huawei Co., Ltd.
[¶]Independent Researcher
daoguang@iscas.ac.cn; yuailun@pku.edu.cn

ABSTRACT

When human programmers have mastered a programming language, it would be easier when they learn a new programming language. In this report, we focus on exploring whether programming languages can boost each other during the instruction fine-tuning phase of code large language models. We conduct extensive experiments of 8 popular programming languages (Python, JavaScript, TypeScript, C, C++, Java, Go, HTML) on StarCoder. Results demonstrate that programming languages can significantly improve each other. For example, CODEM-Python 15B trained on Python is able to increase Java by an absolute 17.95% pass@1 on HumanEval-X. More surprisingly, we found that CODEM-HTML 7B trained on the HTML corpus can improve Java by an absolute 15.24% pass@1. Our training data is released at https://github.com/NL2Code/CodeM.

Keywords Large Language Model · Code Generation · Programming Language · Instruction Tuning

1 Introduction

Code large language models (code LLMs) are blooming recently [Zan et al., 2023]. A lot of code LLMs are released in succession, e.g., Codex [Chen et al., 2021], AlphaCode [Li et al., 2022], PaLM-Coder [Chowdhery et al., 2022], CodeGen [Nijkamp et al., 2023], CodeGeeX [Zheng et al., 2023], StarCoder [Li et al., 2023], and Code Llama [Rozière et al., 2023]. Owing to their amazing code generation performance, code LLMs have attracted considerable attention from both academic and industrial circles. Recent works [Ouyang et al., 2022] have witnessed the instruction tuning technique that can teach LLMs how to follow instructions. In the realm of code generation, WizardCoder [Luo et al., 2023] and PanGu-Coder2 [Shen et al., 2023] also adopt this technique to elicit their code generation capabilities. Although some code LLMs, such as CodeGen-Multi Nijkamp et al. [2023] and StarCoder-base Li et al. [2023], are trained on corpora spanning multiple programming languages, the interplay among these languages remains unexplored. In programming practice, once a human programmer has mastered a programming language, it would be easier to learn a new one due to the homogeneity between programming languages. Motivated by this, we would like to explore whether different programming languages can boost each other during instruction fine-tuning of code LLMs.

To explore this idea, we craft the training corpus for each of 8 popular programming languages (Python, JavaScript, TypeScript, C, C++, Java, Go, HTML), where each language includes about 9K programming exercises. We train StarCoder 7B using the instruction tuning technique on each programming language corpus separately, and test the performance of each fine-tuned model across every programming language. Our findings reveal that programming languages can significantly boost each other. Meanwhile, we found that the improvement margin of different programming languages to each other is related to the language similarity between them. For example, CODEM-JavaScript 7B trained on JavaScript data can yield an absolute 11.80% pass@1 improvement in TypeScript. More interestingly,

[*]The first two authors contributed equally to this work.

```
# Instruction
Design a web page that displays a message when loaded. The message should be "Hello, World". Use HTML code
to achieve this.

# Response
<!DOCTYPE html>
<html>
<head>
    <title>Hello World</title>
</head>
<body>
    <h1>Hello, World</h1>
</body>
</html>
```

Figure 1: A HTML training example of our crafted instruction-answer pairs.

CODEM-HTML 7B trained on the markup language HTML also can achieve an absolute 15.24% pass@1 improvement in Java.

In a nutshell, our contributions can be listed as follows: (1) Our findings suggest that programming languages can significantly boost each other during code LLMs' instruction fine-tuning phase. (2) We glean valuable insights on the correlation between multiple programming languages, paving the way for future research on code generation. (3) We will make our training data publicly available.

2 Methodology

2.1 Crafting Training Corpus of Eight Programming Languages

We select 8 popular programming languages and construct their training data separately. Our selected languages include Python, JavaScript, TypeScript, C, C++, Java, Go, and HTML, covering diverse types such as procedure-oriented, object-oriented, script, and even markup languages. For each programming language, we construct its training data containing about 9K data pairs. Each pair includes both an instruction describing the programming problem and its corresponding response. One practical example of HTML is shown in Figure 1.

Based on these selected languages, we construct a series of monolingual datasets. We start from the dataset of CodeAlpaca 20K[2], and extract those Python-related data to form our seed instruction set. Then for each selected programming language, we evolve existent instructions in the seed instruction set to get corresponding new ones by prompting OpenAI's GPT-3.5[3]. For all the selected languages except HTML, we adopt an in-depth evolution [Xu et al., 2023], by asking GPT-3.5 to rewrite the seed instruction (Python) into a more complicated version relevant to the target language (Python, JavaScript, TypeScript, C, C++, Java, or Go). However, for HTML, we adopt in-breadth evolution to produce a brand-new HTML-related instruction, since HTML (markup language) is too different from other languages (non-markup languages).

2.2 Instruction Tuning

Code pre-trained models such as Codex [Chen et al., 2021] and StarCoder [Li et al., 2023] store a wealth of code knowledge. However, these models only support left-to-right code generation based on context, as they are trained solely on plain code snippets. Of late, the instruction tuning techniques [Ouyang et al., 2022, Luo et al., 2023, Shen et al., 2023] are proposed, which can enhance the model's capabilities of following instructions so as to enable chat features. During instruction tuning, we train StarCoder using the prompt in Figure 2 to obtain our CODEM. We use DeepSpeed to accelerate the training of CODEM with `fp16` enabled. Additionally, we set the batch size to 2 per GPU, the learning rate to 2e-5 with a cosine annealing schedule, the gradient accumulation steps to 4, and the warmup steps to 30. After instruction tuning, we use the prompt in Figure 3 to do the inference on downstream tasks across various programming languages. For inference, we adopt the greedy decoding strategy for sampling. Given that CODEM is a

[2] `https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k`

[3] `https://platform.openai.com/docs/models/gpt-3-5`

```
Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

### Instruction:
{problem}

### Response:
{response}
```

Figure 2: Prompt format of instruction tuning. `{problem}` and `{response}` refer to the instruction and answer obtained in Section 2.1.

```
Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Finish the {language} code for this problem:
{problem}

### Response:
{signature}
```

Figure 3: Prompt format of inference. `{language}`, `{problem}`, and `{signature}` represent the downstream programming language, the given programming problem, and the function header, respectively.

chat-style model, the responses it generates often contain elements beyond just codes, which typically makes them non-executable. So, we extract the code snippets from the generated response to evaluate the performance of code generation.

3 Experiments

3.1 Evaluation Setup

3.1.1 Benchmarks and Baselines

We use HumanEval-X [Zheng et al., 2023] to evaluate the multilingual abilities of models in Python, JavaScript, C++, Java, and Go. HumanEval-X is crafted by adapting HumanEval [Chen et al., 2021] (Python) to other programming languages. Following the same approach as HumanEval-X, we also create two new versions of HumanEval: HumanEval-C and HumanEval-TypeScript. Note that HumanEval can not directly be adapted to markup languages such as HTML, so our downstream evaluation languages do not include HTML.

The primary baseline for all language versions of CODEM is their base model StarCoder. We analyze whether CODEM trained on language A can improve language B, in which case the baselines are CODEM directly trained on language B.

3.1.2 Metrics

We adopt pass@1 as our metric to evaluate all the models. Each model generates one answer using the greedy decoding strategy for each programming task, and the answer would be executed upon the given test cases. Only when all the test cases are passed, the programming task can be considered solved with the generated code. In this setting, pass@1 can be formulated as $\frac{|P_c|}{|P|}$, where $|P|$ denotes the total number of programming tasks in HumanEval and $|P_c|$ represents the number of solved tasks. In essence, the pass@1 metric we use can be considered as the accuracy.

3.2 Results

3.2.1 Main Results

Table 1 shows the performance of CODEM, which are a series of models trained on monolingual datasets of eight languages respectively, across different language versions of HumanEval. As we can see, all CODEM models outperform

Table 1: Pass@1 (Accuracy) of StarCoder 7B and CODEM trained on various programming languages. The numbers in red represent the absolute increase compared to StarCoder 7B.

Model	HumanEval-Multilingual						
	Python	JavaScript	TypeScript	C	C++	Java	Go
StarCoder 7B	26.83	24.39	28.57	24.69	25.61	23.17	24.39
CODEM-Python	38.41$^{11.58}$	34.76$^{10.37}$	33.54$^{4.97}$	29.01$^{4.32}$	34.15$^{8.54}$	37.20$^{14.03}$	27.44$^{3.05}$
CODEM-JavaScript	37.20$^{10.37}$	**40.24**$^{15.85}$	**40.37**$^{11.80}$	27.78$^{3.09}$	32.93$^{7.32}$	34.76$^{11.59}$	26.22$^{1.83}$
CODEM-TypeScript	33.54$^{6.71}$	37.80$^{13.41}$	37.27$^{8.70}$	30.25$^{5.56}$	30.49$^{4.88}$	28.05$^{4.88}$	25.61$^{1.22}$
CODEM-C	39.63$^{12.8}$	37.20$^{12.81}$	32.30$^{3.73}$	32.10$^{7.41}$	35.37$^{9.76}$	38.41$^{15.24}$	28.66$^{4.27}$
CODEM-C++	34.57$^{7.74}$	35.37$^{10.98}$	32.30$^{3.73}$	**34.57**$^{9.80}$	**39.02**$^{13.41}$	37.20$^{14.03}$	28.05$^{3.66}$
CODEM-Java	35.37$^{8.54}$	33.54$^{9.15}$	32.30$^{3.73}$	29.63$^{4.94}$	31.10$^{5.49}$	37.80$^{14.63}$	27.44$^{3.05}$
CODEM-Go	35.98$^{9.15}$	33.54$^{9.15}$	31.68$^{3.11}$	30.25$^{5.56}$	34.15$^{8.54}$	35.98$^{12.81}$	**32.32**$^{7.93}$
CODEM-HTML	31.71$^{4.88}$	33.54$^{9.15}$	32.30$^{3.73}$	25.93$^{1.24}$	28.66$^{3.05}$	38.41$^{15.24}$	28.05$^{3.66}$
CODEM-Mixed	**43.29**$^{16.46}$	37.20$^{12.81}$	37.89$^{9.32}$	32.10$^{7.41}$	37.80$^{12.19}$	**39.63**$^{16.46}$	29.27$^{4.88}$

Table 2: Pass@1 of StarCoder 15B and CODEM-Python. The numbers in red denote the absolute improvement compared to StarCoder 15B.

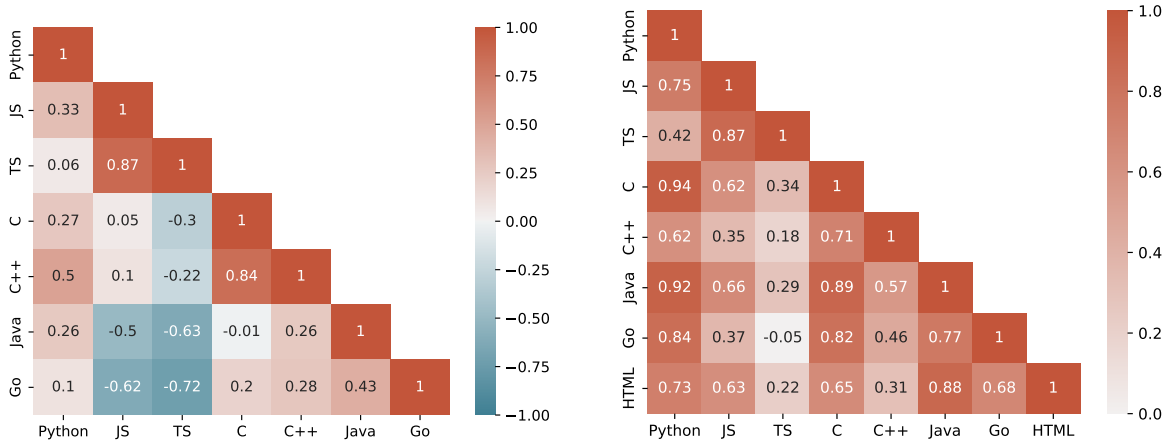
Model	HumanEval-Multilingual						
	Python	JavaScript	TypeScript	C	C++	Java	Go
StarCoder 15B	32.93	30.79	32.29	26.99	31.55	30.22	17.61
CODEM-Python	64.63$^{31.70}$	47.56$^{16.77}$	39.75$^{7.46}$	35.19$^{9.20}$	43.80$^{12.35}$	48.17$^{17.95}$	34.76$^{17.15}$

their base model StarCoder 7B across all programming languages by a large margin. Also, we found that programming languages can boost each other significantly. For example, CODEM-Python trained solely on Python corpus is able to improve HumanEval-Java by an absolute 14.03% pass@1. This finding reveals the inherent commonalities among different programming languages. More surprisingly, CODEM-HTML boosts HumanEval-Java by an absolute 15.24% pass@1, even exceeding CODEM-Java. Similarly, CODEM-C++ beats CODEM-C on HumanEval-C, and CODEM-JavaScript beats CODEM-TypeScript on HumanEval-Typescript. Drawing upon these observations, we conjecture that the improvement in multilingual code generation performance is predominantly due to instruction tuning unlocking the model's inherent potential, such as natural or programming language understanding and following-instruction capabilities, rather than merely incorporating new knowledge. In addition to training CODEM on a monolingual training corpus, we further construct a 9K multilingual training set covering 8 programming languages. Although each language comprises only a small amount (~1.2K) of training instances, experimental findings suggest that CODEM-Mixed excels in all languages, even surpassing CODEM-Python on HumanEval-Python and CODEM-Java on HumanEval-Java. This suggests that it is possible to yield superior code generation performance by leveraging multilingual data in instruction tuning, without harming the generalization of the model.

We also conduct experiments on StarCoder 15B to verify the effectiveness of CODEM. Specifically, we obtain 108K Python training data following WizardCoder [Luo et al., 2023], and finetune StarCoder 15B to get CODEM-Python. The results are shown in Table 2. CODEM-Python achieves state-of-the-art performance on HumanEval-Python with 64.63% pass@1, compared with other models of the same scale. CODEM-Python also gets a tremendous improvement in the generation of other programming languages. For instance, it improves Java and JavaScript by an absolute 17.95% and 16.77% pass@1, respectively.

3.2.2 Closer Analysis

We analyze the correlation between different programming languages. As illustrated in Figure 4 (a), the improvement of code generation performance is sensitive to training corpus of different programming languages. Moreover, we found that C and C++ can boost each other more significantly, which is the same for JavaScript and TypeScript. It is reasonable because these languages are correlated to each other in language design, sharing some common syntax and grammar. Figure 4 (b) shows that training on each programming language can boost the code generation performance of all other languages. We can see that the correlation values in Figure 4 (b) are mostly all positive, implying that the improvement trend of different language brought by one monolingual training corpus is relatively similar.



(a) (b)

Figure 4: Correlations between different programming languages. We regard the data in Table 1 as a matrix, and use "`df.corr()`" from the Pandas library to compute the correlation between different programming languages. The correlation results before and after "`df.T`" are presented in (a) and (b), respectively.

4 Related Work

Codex [Chen et al., 2021] with 12-billion parameters is able to solve Python programming problems automatically. This remarkable success triggered a significant buzz in both the academic and industrial realms. Followed by Codex, a plenty of code LLMs are proposed, including AlphaCode [Li et al., 2022], PaLM-Coder [Chowdhery et al., 2022], CodeGen [Nijkamp et al., 2023], InCoder [Fried et al., 2023], CodeGeeX [Zheng et al., 2023], replit[4], CodeT5 [Wang et al., 2021, 2023], PyCodeGPT [Zan et al., 2022], SantaCoder [Allal et al., 2023], StarCoder [Li et al., 2023], Code Llama [Rozière et al., 2023], and phi-1 [Gunasekar et al., 2023]. These above models are trained on a large-scale code corpus and achieve impressive code generation performance. During their pre-training, some models are trained on datasets of multilingual programming languages and then fine-tuned on a monolingual dataset to produce a more powerful specialist version. As for the instruction fine-tuning phase, WizardCoder [Luo et al., 2023], PanGu-Coder2 [Shen et al., 2023], and Phind-CodeLlama[5] are proposed to bolster the capability of following instructions and further boost the code generation capability. Yet, none of these aforementioned models explore the intricate interplay between different programming languages. In this report, we therefore would like to investigate whether training code LLMs on monolingual data can bolster performance in other programming languages.

5 Conclusion

Our findings reveal that a monolingual training corpus can enhance the multilingual code generation capabilities of code LLMs via instruction tuning. This highlights the intrinsic commonality and interconnectedness among multiple programming languages. In our future work, we plan to delve into the reasons why multiple languages can enhance each other. Also, we will explore how to leverage our findings to elevate code generation capabilities for these obscure or less-used programming languages by training on data from those popular ones.

Acknowledgements

We would like to thank our colleagues for their valuable feedback and insights. Special thanks to An Fu (Huawei), Jingyang Zhao (Huawei), and Yuenan Guo (Huawei) for their constructive help throughout this research.

References

Daoguang Zan, Bei Chen, Fengji Zhang, Dianjie Lu, Bingchao Wu, Bei Guan, Wang Yongji, and Jian-Guang Lou. Large language models meet NL2Code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for*

[4] `https://huggingface.co/replit/replit-code-v1-3b`
[5] `https://huggingface.co/Phind/Phind-CodeLlama-34B-v1`

Computational Linguistics (Volume 1: Long Papers), pages 7443–7464, Toronto, Canada, July 2023. Association for Computational Linguistics. URL https://aclanthology.org/2023.acl-long.411.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, S. Arun Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.

Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom, Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de, Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey, Cherepanov, James Molloy, Daniel Jaymin Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de, Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378:1092 – 1097, 2022.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam M. Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Benton C. Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier García, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Díaz, Orhan Firat, Michele Catasta, Jason Wei, Kathleen S. Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling language modeling with pathways. *ArXiv*, abs/2204.02311, 2022.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. CodeGen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shanshan Wang, Yufei Xue, Zi-Yuan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. CodeGeeX: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *ArXiv*, abs/2303.17568, 2023.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder: may the source be with you!, 2023.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. Code Llama: Open foundation models for code, 2023.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke E. Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Francis Christiano, Jan Leike, and Ryan J. Lowe. Training language models to follow instructions with human feedback. *ArXiv*, abs/2203.02155, 2022.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. WizardCoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. PanGu-Coder2: Boosting large language models for code with ranking feedback, 2023.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. WizardLM: Empowering large language models to follow complex instructions, 2023.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations*, 2023.

Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8696–8708, 2021.

Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. Codet5+: Open code large language models for code understanding and generation, 2023.

Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. CERT: Continual pre-training on sketches for library-oriented code generation. In *International Joint Conference on Artificial Intelligence*, 2022.

Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Muñoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alexander Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, J. Poirier, Hailey Schoelkopf, Sergey Mikhailovich Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Franz Lappert, Francesco De Toni, Bernardo Garc'ia del R'io, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luisa Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Christopher Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. SantaCoder: don't reach for the stars! *ArXiv*, abs/2301.03988, 2023.

Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need, 2023.